

Adversarial Machine Learning Homework Assignment 1

The assignment is due at the end of the day on Thursday, January 29.

Objective:

- Implement white-box evasion attacks against deep learning-based classification models.

Dataset: For this assignment, we will use the Vehicles dataset. The dataset consists of images with 7 categories of vehicles, and it has 5,590 images in total. Examples of images from the Vehicles dataset are shown in Figure 1. The categories include: Auto Rickshaws (800 images), Bikes (800 images), Cars (790 images), Motorcycles (800 images), Planes (800 images), Ships (800 images), and Trains (800 images).

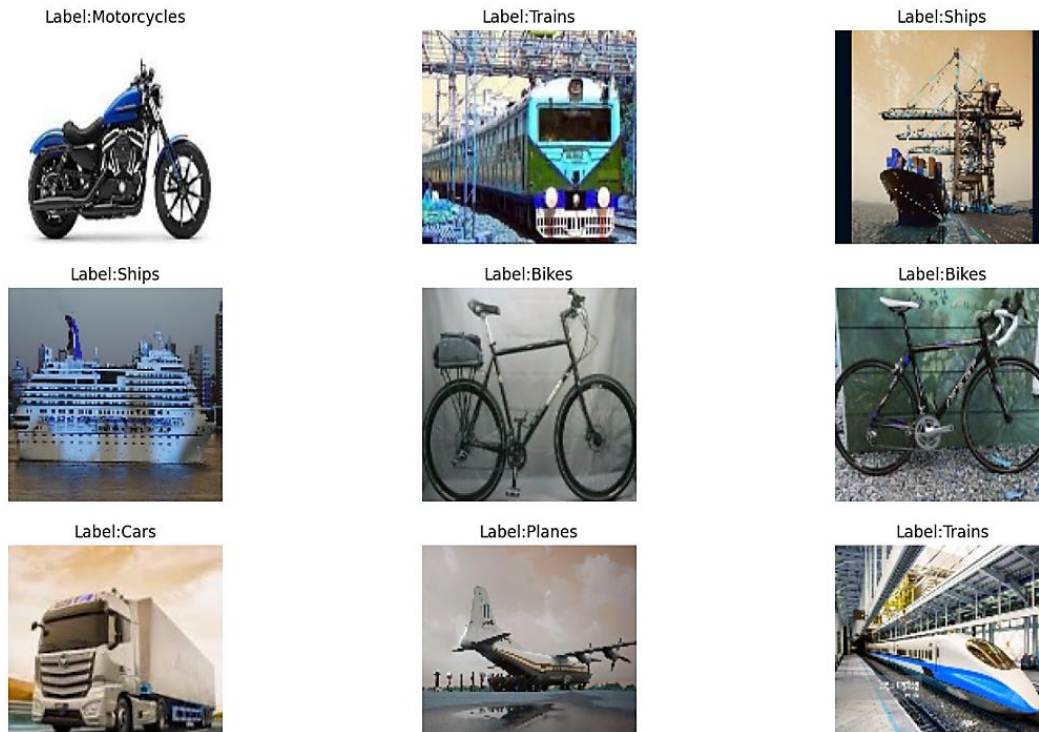


Figure 1. Example images from the Vehicles dataset.

FGSM and PDG Attacks against TensorFlow Image Classification Models

A starter code for loading the dataset is provided with this assignment. Please use the provided file to load the dataset.

Task 1: Train a deep-learning model for classification of the Vehicles dataset using the TensorFlow library.

For GPU access, it is recommended to use Google Colab Pro. Note that Google provides free access to Colab Pro for students. Please check the information on Canvas in the Modules section, and follow the link to request access. You will need to use your vandal email to prove that you are student. However, if you have access to GPU from other sources, it is ok to use it.

For solving the tasks in this assignment, please follow the sample code for White Box Evasion Attacks with TensorFlow posted on Canvas in the 'Modules/Codes' section.

Similarly to the provided sample code on Canvas, adopt a pretrained VGG-16 or ResNet (e.g., ResNet50) deep neural network model, and add a classifier head on top of the pretrained backbone.

Perform hyperparameter tuning to obtain accuracy on the test dataset above 95%.

If you haven't worked with Jupyter Notebooks or Google Colab, please review these tutorials from my course Applied Data Science with Python:

- Tutorial 1 - Working with Jupyter Notebooks ([link](#))
- Tutorial 5 - Google Colab ([link](#))

Also, for the students who don't have previous background in Artificial Neural Networks, I suggest reviewing these lectures from my course:

- Lecture 15 - Convolutional Neural Networks with Keras-TensorFlow ([link](#))
- Lecture 16 - Model Selection, Hyperparameter Tuning ([link](#)).

If needed, please review other lectures from the course.

Estimated running time: between 5 and 20 minutes using a GPU.

Note: If the model training takes hours instead of less than 20 minutes, it is possible that you are not using the GPU. Please check the above tutorial on Google Colab to learn how to enable GPU.

Report (40 marks):

- Fill in Table 1 with the values for the classification accuracy for the training set, validation set, and test set of images. For full marks, it is expected to report a test accuracy above 95%. You don't need to report other performance metrics, since the focus is on adversarial attacks.
- Plot the training and validation loss and accuracy curves (similar to the example in Figure 2 below).

If applicable, provide any other observations regarding the model or the dataset.

Table 1. Classification accuracy.

Model	Train Set	Validation Set	Test Set

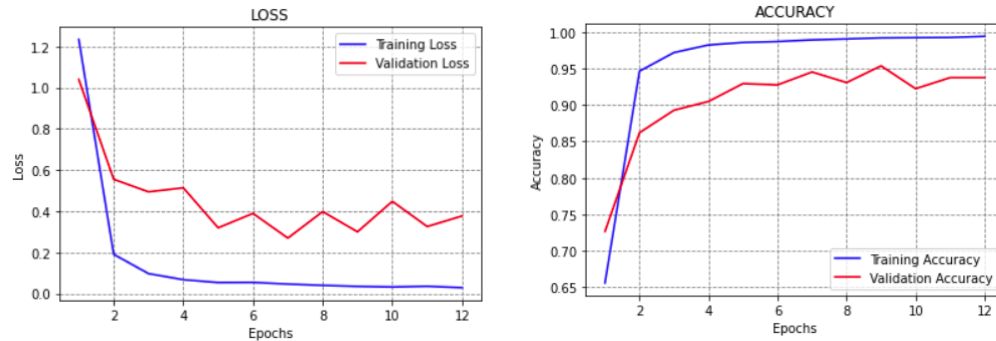


Figure 2. Loss and accuracy plots for a DL model. This is an example, and not the actual expected plot.

Task 2: Implement non-targeted white-box evasion attacks against the deep learning model from Task 1.

The attacks to be implemented are Fast Gradient Sign Method (FGSM) and Projected Gradient Descent (PGD).

Use the [Adversarial Robustness Toolbox](#) (ART) for implementing the attacks for this assignment. Similar to Task 1, if you follow the provided code in the White Box Evasion Attacks with TensorFlow notebook posted on Canvas in the 'Modules/Codes' section, solving this part of the assignment should be straightforward.

If you are interested to learn more about the ART toolbox, the ART library provides many [notebooks](#) with explanations on how to use the various attacks and defenses in the library.

Apply FGSM and PGD attacks to create non-targeted adversarial examples using the first 200 images of the test set. Apply the following perturbation magnitudes: $\epsilon = [1/255, 3/255, 5/255, 8/255, 20/255, 50/255, 80/255]$. Plot the overall accuracy of the model versus the perturbation size ϵ (e.g., the expected plot should look like Figure 3, note that $\epsilon = 80/255 \approx 0.3$). For the FGSM attack, plot the first clean test image and the adversarial images with added adversarial perturbation of $\epsilon = [3/255, 8/255, 20/255, 50/255, 80/255]$, and display the predicted label (the figure should look similar to Figure 4 below). When you run the codes, you will understand better why FGSM is called a fast method.

Estimated time: between 15 and 30 minutes.

Report (60 marks):

- Fill in Table 2 with the values for the classification accuracy for the clean images and perturbed images, with perturbations levels of $\epsilon = 1/255, 5/255$, and $8/255$.
- Plot the accuracy versus perturbation ϵ for FGSM and PGD adversarial attacks (similar to the example in Figure 3).
- Plot figures with added adversarial perturbation and the labels for $\epsilon = [3/255, 8/255, 20/255, 50/255, 80/255]$, similar to the examples in Figure 4.

- d) Explain the working principles of FGSM and PGD attacks using between 4 and 8 sentences for each of the attacks. Provide an analysis of the results. Explain the meaning of the parameter ϵ and its impact on the results. Explain which of these two attacks is more efficient, and why?

Table 2. Classification accuracy on clean and adversarial images.

Model	Clean images	Adversarial images $\epsilon=1/255$	Adversarial images $\epsilon=5/255$	Adversarial images $\epsilon=8/255$
FGSM attack				
PGD attack				

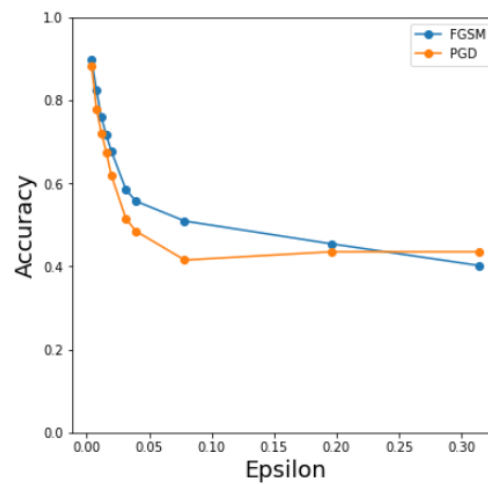
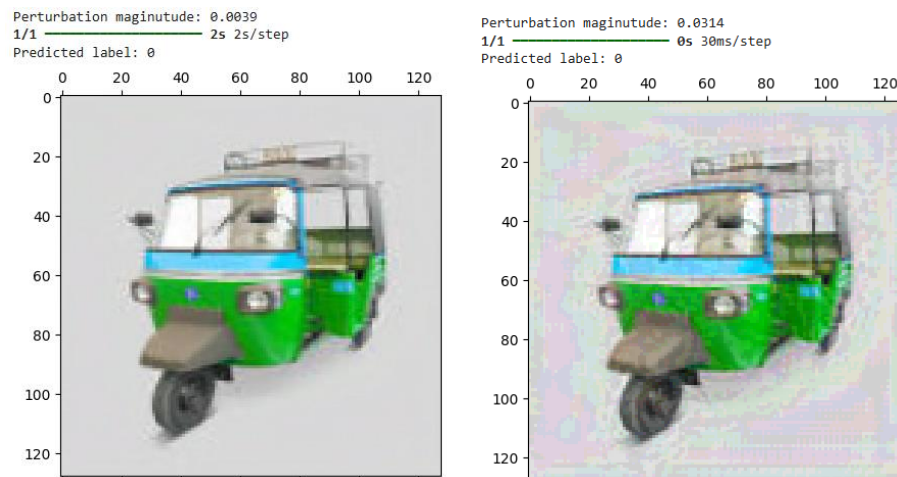


Figure 3. Plots of accuracy versus perturbation. This is an example figure, and not the plot for this task.



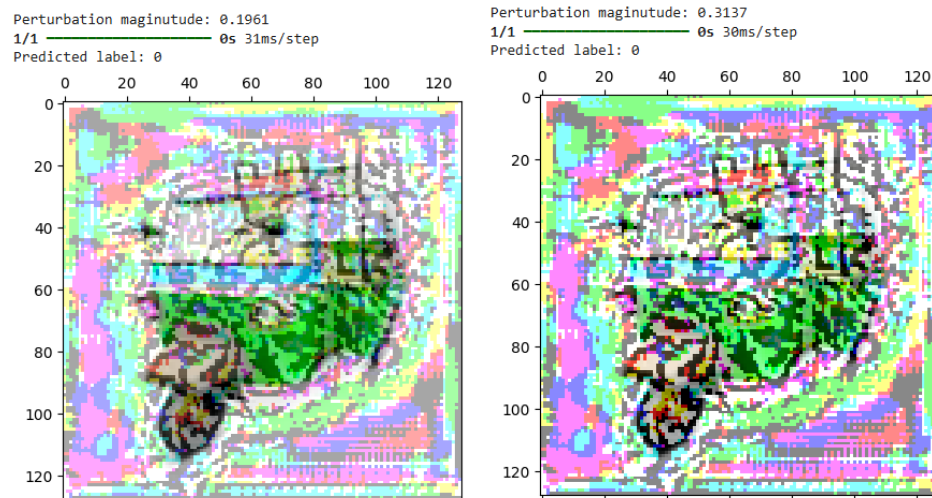


Figure 4. Examples of adversarial images.

Submission Documents

The assignment documents are submitted on Canvas. Note that it is possible to submit multiple files at the same time, just drag-and-drop the files or attach all the files while the submit window is open.

1. Submit all your codes as Jupyter Notebooks. Please try to comment your codes extensively, introduce the names of all used variables, avoid one-letter variables, etc., to improve the readability of the codes. For instance, for this assignment you can submit one single Jupyter Notebook with the codes. You don't need to submit the dataset on Canvas.
2. Prepare a brief report with tables, graphs, and results: the report can be prepared either as a separate MS Word/PDF file, or it can be integrated into your Jupyter Notebooks by typing your analysis directly into text cells in the Jupyter Notebooks.